

Fundamentele limbajelor de programare

Rezolvări pentru exercițiile si demonstrațiile lăsate la curs

Pregătire examen

Ce conține PDF-ul. Am adunat exercițiile explicate din cursuri si rezultatele marcate ca "lăsate ca exercițiu", "se demonstrează ușor", "demonstrabile prin inducție" sau folosite ca demonstrații omise. Pentru fiecare ai enunț, rezolvare completă si, unde este util, o variantă scurtă de scris la examen.

Materiale acoperite. C1 Preliminarii matematice; C2/C2-cont Semantică operațională; C3/C3-cont Logică Hoare si wp; C4/C4-cont Prolog si unificare; C5/C5-cont Rezoluție; C6/C6-cont Lambda-calcul, confluență si standardizare.

Observație. Pentru teoremele mari, cum ar fi Church-Rosser sau completitudinea rezoluției, cursul dă de obicei doar schiță. Aici am scris demonstrații la nivel de examen: suficient de complete pentru a explica mecanismul, fără a transforma PDF-ul într-un tratat de zeci de pagini pe o singură teoremă.

Contents

Cum să folosești documentul	3
1 C1 - Preliminarii matematice	4
1.1 Relații funcționale si grafice	4
1.2 Domeniul unui grafic este determinat de relație	4
1.3 O funcție este inversabilă dacă si numai dacă este bijectivă	4
1.4 Principiul recursiei pe formule - unicitatea funcției	5
1.5 Principiul recursiei pe formule - existența valorii	5
1.6 Principiul recursiei pe formule - cazurile omise din unicitatea graficului	6
2 C2 - Semantică operațională IMP	8
2.1 Determinismul semanticii big-step	8
2.2 Dacă un pas small-step este urmat de big-step, atunci big-step-ul original există	8
2.3 Corespondența small-step / big-step pentru programe terminate	9
3 C3 - Logică Hoare si cea mai slabă precondiție	11
3.1 Regula pentru if b then S fără else	11
3.2 Lema substituției pentru axioma atribuirii	11
3.3 Echivalența dintre triplet Hoare semantic si wls	12
3.4 Când execuția termină mereu, $ws = wls$	13
3.5 Exercițiul din demonstrația $wp(\text{while } bS, B)^I = ws_I(\text{while } bS, B)$	13
4 C4 - Unificare	15
4.1 Lema ajutoare pentru regula REZOLVĂ	15
4.2 Corectitudinea invariantului pentru SCOATE	15
4.3 Corectitudinea invariantului pentru DESCOMPUNE	16
4.4 Corectitudinea invariantului pentru REZOLVĂ	16
4.5 Varianta algoritmului de unificare si terminarea	17
4.6 Corectitudinea parțială si totală a algoritmului de unificare	17
5 C5 - Prolog, rezoluție, semantică	19
5.1 Termeni, formule atomice si enunțuri pentru alfabetul dat	19
5.2 Arborele SLD pentru $?- p(X, X)$	19
5.3 Proprietăți ale satisfacției pentru mulțimi de enunțuri	21
5.4 Proprietăți ale deducției semantice	21
5.5 Proprietățile substituției în logica de ordinul I	21

5.6	De ce $B_\sigma \models_H P$ rămâne adevărat	22
5.7	Corectitudinea rezoluției SLD - demonstrație de examen	23
6	C6 - Lambda-calcul	24
6.1	Eliminarea parantezelor inutile	24
6.2	Adăugarea parantezelor după convenții	24
6.3	Substituții lambda - exemplul 1	25
6.4	Substituții lambda - exemplul 2, cu redenumire pentru evitarea capturării	25
6.5	Substituții lambda - exemplul 3, captură evitată	26
6.6	Substituții lambda - exemplul 4	26
6.7	Termeni care pot fi aduși la formă normală	26
6.8	Incluziunile $\rightarrow_\beta \subseteq \rightarrow_\parallel$ și $\rightarrow_\parallel \subseteq \rightarrow_\beta^*$	27
6.9	Comutativitatea substituției	28
6.10	Lemele de formă pentru reducerea paralelă	29
6.11	Substituția păstrează reducerea paralelă	29
6.12	De ce Church-Rosser dă unicitatea formei normale	30
6.13	Proprietăți ale reducerii standard	30
6.14	Lema: dacă $L \rightarrow_{st} (\lambda x.M)N$, atunci $L \rightarrow_{st} M[x := N]$	31
6.15	Absorbția unei beta-reduceri într-o reducere standard	31
6.16	Teorema de standardizare - demonstrație de examen	32
	Checklist final pentru examen	33

Cum să folosești documentul

Pentru examen, învață în această ordine:

1. Logică Hoare și wp: aici apar demonstrații cu substituție și calcul “înapoi”.
2. Unificare: algoritmul, invariantul, terminarea și corectitudinea.
3. Rezoluție SLD: arbore, respingere, substituție calculată, corectitudine.
4. Lambda-calcul: substituție fără captură, α -echivalență, β -reducere, formă normală, standardizare.
5. Preliminarii: funcții, grafice, inversă, recursie structurală.

La fiecare exercițiu, partea “Variantă scurtă de scris” este forma compactă pe care o poți reproduce pe foaie. Partea “Rezolvare” este pentru înțelegere.

1 C1 - Preliminarii matematice

1.1 Relații funcționale și grafice

Enunț. Fie R o relație binară. Sunt echivalente:

- (i) există mulțimi A, B astfel încât R este grafic între A și B ;
- (ii) pentru orice x, y, z , dacă $(x, y), (x, z) \in R$, atunci $y = z$.

Ideea de examen. Un grafic de funcție înseamnă: pentru fiecare element din domeniu există exact o imagine. Dacă R nu trimite același x în două valori diferite, putem lua domeniul exact ca mulțimea primelor componente din R și codomeniul ca mulțimea componentelor a doua.

Rezolvare. Presupunem mai întâi că există A, B astfel încât R este grafic între A și B . Prin definiție, pentru orice $a \in A$ există un unic $b \in B$ cu $(a, b) \in R$. Dacă $(x, y), (x, z) \in R$, atunci $x \in A$, iar y și z sunt două valori din B asociate aceluiasi x . Prin unicitate, $y = z$.

Invers, presupunem proprietatea funcțională:

$$(x, y), (x, z) \in R \implies y = z.$$

Definim

$$A := \{x \mid \exists y (x, y) \in R\}, \quad B := \{y \mid \exists x (x, y) \in R\}.$$

Atunci $R \subseteq A \times B$. Pentru orice $a \in A$, din definiția lui A există b cu $(a, b) \in R$, iar acel b este în B . Dacă mai există b' cu $(a, b') \in R$, atunci, aplicând ipoteza la (a, b) și (a, b') , obținem $b = b'$. Deci pentru fiecare $a \in A$ există exact un $b \in B$ cu $(a, b) \in R$, adică R este grafic între A și B .

Cazul $R = \emptyset$ este inclus: atunci $A = B = \emptyset$, iar condiția de totalitate pe A este adevărată vid. $\square$

Variantă scurtă de scris. “ $\implies$ ” rezultă din unicitatea valorii într-un grafic. “ $\Leftarrow$ ”: iau $A = \text{Dom}(R)$ și $B = \mathfrak{S}(R)$. Existența valorii vine din definiția domeniului, iar unicitatea vine din ipoteza $(x, y), (x, z) \in R \implies y = z$. Deci R este grafic între A și B .

1.2 Domeniul unui grafic este determinat de relație

Enunț. Fie R relație binară și A, B, C, D astfel încât R este grafic atât între A și B , cât și între C și D . Atunci $A = C$.

Rezolvare. Arătăm dubla incluziune.

Fie $a \in A$. Pentru că R este grafic între A și B , există $b \in B$ cu $(a, b) \in R$. Deoarece R este și grafic între C și D , avem $R \subseteq C \times D$. Din $(a, b) \in R \subseteq C \times D$ rezultă $a \in C$. Deci $A \subseteq C$.

Analog, fie $c \in C$. Pentru că R este grafic între C și D , există $d \in D$ cu $(c, d) \in R$. Cum $R \subseteq A \times B$, rezultă $c \in A$. Deci $C \subseteq A$.

Prin urmare $A = C$. $\square$

1.3 O funcție este inversabilă dacă și numai dacă este bijectivă

Enunț. Fie $f : A \rightarrow B$. Atunci f este inversabilă dacă și numai dacă f este bijectivă.

Rezolvare. ($\Rightarrow$) Presupunem că f este inversabilă. Există $g : B \rightarrow A$ astfel încât

$$g \circ f = \text{id}_A, \quad f \circ g = \text{id}_B.$$

Arătăm că f este injectivă. Fie $x, y \in A$ cu $f(x) = f(y)$. Aplicăm g :

$$g(f(x)) = g(f(y)).$$

Dar $g(f(x)) = x$ și $g(f(y)) = y$, deci $x = y$. Prin urmare f este injectivă.

Arătăm că f este surjectivă. Fie $b \in B$. Punem $a := g(b) \in A$. Atunci

$$f(a) = f(g(b)) = (f \circ g)(b) = \text{id}_B(b) = b.$$

Deci orice $b \in B$ are preimagine. f este surjectivă. Rezultă că f este bijectivă.

($\Leftarrow$) Presupunem că f este bijectivă. Pentru fiecare $b \in B$, prin surjectivitate există cel puțin un $a \in A$ cu $f(a) = b$, iar prin injectivitate acel a este unic. Definim

$$g : B \rightarrow A, g(b) = \text{unicul } a \in A \text{ cu } f(a) = b.$$

Atunci pentru orice $a \in A$, $g(f(a)) = a$, deci $g \circ f = \text{id}_A$. Pentru orice $b \in B$, prin definiția lui g , $f(g(b)) = b$, deci $f \circ g = \text{id}_B$. Așadar g este inversa lui f , deci f este inversabilă. $\square$

1.4 Principiul recursiei pe formule - unicitatea funcției

Enunț. Fie A o mulțime și funcțiile $G_0 : V \rightarrow A$, $G_{\neg} : A \rightarrow A$, $G_{\rightarrow} : A^2 \rightarrow A$. Există cel mult o funcție $F : \text{Form} \rightarrow A$ cu proprietățile:

$$F(v) = G_0(v), \quad F(\neg\varphi) = G_{\neg}(F(\varphi)), \quad F(\varphi \rightarrow \psi) = G_{\rightarrow}(F(\varphi), F(\psi)).$$

Rezolvare. Presupunem că $F, H : \text{Form} \rightarrow A$ satisfac aceleași ecuații. Trebuie să arătăm că $F = H$, adică $F(\varphi) = H(\varphi)$ pentru orice formulă φ .

Folosim inducție structurală după forma lui φ .

Caz 1: $\varphi = v$, unde $v \in V$. Atunci

$$F(v) = G_0(v) = H(v).$$

Caz 2: $\varphi = \neg\psi$. Ipoteza de inducție spune că $F(\psi) = H(\psi)$. Atunci

$$F(\neg\psi) = G_{\neg}(F(\psi)) = G_{\neg}(H(\psi)) = H(\neg\psi).$$

Caz 3: $\varphi = \psi \rightarrow \chi$. Ipotezele de inducție sunt $F(\psi) = H(\psi)$ și $F(\chi) = H(\chi)$. Deci

$$F(\psi \rightarrow \chi) = G_{\rightarrow}(F(\psi), F(\chi)) = G_{\rightarrow}(H(\psi), H(\chi)) = H(\psi \rightarrow \chi).$$

Prin inducție structurală, $F(\varphi) = H(\varphi)$ pentru orice formulă φ . Deci funcția este unică. $\square$

1.5 Principiul recursiei pe formule - existența valorii

Enunț. În construcția mulțimii $S \subseteq \text{Form} \times A$, arătați prin inducție structurală că pentru orice $\varphi \in \text{Form}$ există $a \in A$ cu $(\varphi, a) \in S$.

Rezolvare. Definim proprietatea

$$P(\varphi) : \exists a \in A \text{ astfel încât } (\varphi, a) \in S.$$

Arătăm $P(\varphi)$ pentru toate formulele, prin inducție structurală.

Caz 1: $\varphi = v$, variabilă propozițională. Prin construcția lui S , pentru orice $v \in V$ avem

$$(v, G_0(v)) \in S.$$

Deci există valoare, anume $a = G_0(v)$.

Caz 2: $\varphi = \neg\psi$. Ipoteza de inducție spune că există $a \in A$ cu $(\psi, a) \in S$. Prin proprietatea de închidere a lui S , rezultă

$$(\neg\psi, G_{\neg}(a)) \in S.$$

Deci există valoare pentru $\neg\psi$.

Caz 3: $\varphi = \psi \rightarrow \chi$. Ipotezele de inducție spun că există $a, b \in A$ cu

$$(\psi, a) \in S, \quad (\chi, b) \in S.$$

Prin proprietatea de închidere a lui S pentru implicație,

$$(\psi \rightarrow \chi, G_{\rightarrow}(a, b)) \in S.$$

Deci există valoare pentru $\psi \rightarrow \chi$.

Prin inducție structurală, orice formulă are cel puțin o valoare asociată în S . $\square$

1.6 Principiul recursiei pe formule - cazurile omise din unicitatea graficului

Enunț. În demonstrația că S este grafic, cursul tratează cazul implicației. Completați cazurile pentru variabilă și negație.

Rezolvare. Notăm cu B mulțimea formulelor φ pentru care există unic $a \in A$ cu $(\varphi, a) \in S$. Vrem $B = \text{Form}$.

Presupunem prin absurd că $B \neq \text{Form}$. Prin contrapusa inducției structurale, unul dintre cazuri eșuează. Cursul tratează cazul implicației. Rămân cazurile:

Cazul variabilă. Există $v \in V$ cu $v \notin B$. Știm că $(v, G_0(v)) \in S$. Pentru că $v \notin B$, există $q \neq G_0(v)$ cu $(v, q) \in S$. Definim

$$T := S \setminus \{(v, q)\}.$$

Arătăm că T încă satisface regulile de construcție ale lui S :

- pentru orice $u \in V$, trebuie $(u, G_0(u)) \in T$. Dacă $u \neq v$, perechea nu este eliminată. Dacă $u = v$, perechea este $(v, G_0(v))$, nu (v, q) , deoarece $q \neq G_0(v)$;
- regulile pentru negație și implicație rămân valide: dacă premisele sunt în T , concluzia era în S ; dacă acea concluzie ar fi exact (v, q) , ar trebui să fie o formulă variabilă, imposibil, fiindcă concluziile acestor reguli au forma $\neg\varphi$ sau $\varphi \rightarrow \psi$.

Deci T are aceleași proprietăți ca S , dar este strict inclusă în S , contradicție cu minimalitatea lui S .

Cazul negație. Există $\varphi_0 \in B$ cu $\neg\varphi_0 \notin B$. Deoarece $\varphi_0 \in B$, există unic a_0 cu $(\varphi_0, a_0) \in S$. Atunci prin regula pentru negație,

$$(\neg\varphi_0, G_{\neg}(a_0)) \in S.$$

Pentru că $\neg\varphi_0 \notin B$, există $q \neq G_{\neg}(a_0)$ cu $(\neg\varphi_0, q) \in S$. Definim

$$T := S \setminus \{(\neg\varphi_0, q)\}.$$

Verificăm închiderea:

- perechile de bază $(v, G_0(v))$ nu sunt eliminate, pentru că au prima componentă variabilă, nu negație;
- pentru regula de negație: dacă $(\psi, a) \in T$, atunci $(\neg\psi, G_{\neg}(a)) \in S$. Dacă această pereche ar fi cea eliminată, atunci $\psi = \varphi_0$ și $G_{\neg}(a) = q$. Dar cum $(\varphi_0, a) \in T \subseteq S$ și valoarea pentru φ_0 este unică, $a = a_0$, deci $G_{\neg}(a) = G_{\neg}(a_0)$, contradicție cu $q \neq G_{\neg}(a_0)$;
- pentru implicație: concluzia are forma $\psi \rightarrow \chi$, deci nu poate fi perechea eliminată, care are prima componentă $\neg\varphi_0$.

Deci T este o submulțime strictă a lui S cu aceleași proprietăți de închidere, contradicție. Prin urmare cazurile variabilă și negație nu pot apărea. $\square$

2 C2 - Semantică operațională IMP

2.1 Determinismul semanticii big-step

Enunț util. Demonstrați că evaluarea big-step pentru IMP este deterministă: expresiile aritmetice au o singură valoare, expresiile booleene au o singură valoare, iar instrucțiunile care termină produc o singură stare finală.

Rezolvare. Se demonstrează simultan, prin inducție după derivare.

Expresii aritmetice. Vrem:

$$\langle a, \sigma \rangle \Downarrow \langle i \rangle \text{ și } \langle a, \sigma \rangle \Downarrow \langle j \rangle \Rightarrow i = j.$$

Cazurile:

- $a = n$. Singura regulă posibilă este (Num), deci $i = n = j$.
- $a = x$. Singura regulă posibilă este (Id), deci $i = \sigma(x) = j$.
- $a = a_1 + a_2$. Din prima derivare avem valori i_1, i_2 pentru subexpresii, cu $i = i_1 + i_2$. Din a doua derivare avem j_1, j_2 , cu $j = j_1 + j_2$. Prin ipoteza de inducție pe a_1 și a_2 , $i_1 = j_1$ și $i_2 = j_2$. Deci $i = j$. Cazurile – și * sunt identice.

Expresii booleene. Vrem:

$$\langle b, \sigma \rangle \Downarrow \langle t_1 \rangle, \quad \langle b, \sigma \rangle \Downarrow \langle t_2 \rangle \Rightarrow t_1 = t_2.$$

Cazurile pentru true și false sunt imediate. Pentru comparații, de exemplu $a_1 \leq a_2$, valorile aritmetice ale lui a_1, a_2 sunt unice prin partea anterioară. Deci adevărul comparației este unic. Pentru $\neg b$, $b_1 \wedge b_2$, $b_1 \vee b_2$, folosim ipotezele de inducție pe subformule.

Instrucțiuni. Vrem:

$$\langle c, \sigma \rangle \Downarrow \langle \sigma_1 \rangle, \quad \langle c, \sigma \rangle \Downarrow \langle \sigma_2 \rangle \Rightarrow \sigma_1 = \sigma_2.$$

Cazuri importante:

- $c = \text{skip}$: ambele stări finale sunt σ .
- $c = x := a$: valoarea lui a este unică, deci ambele stări sunt $\sigma[x \mapsto i]$.
- $c = c_1; c_2$: prima derivare trece printr-o stare intermediară ρ_1 , iar a doua prin ρ_2 . Prin inducție pe c_1 , $\rho_1 = \rho_2$. Apoi prin inducție pe c_2 , stările finale coincid.
- $c = \text{if } b \text{ then } c_1 \text{ else } c_2$: valoarea lui b este unică. Deci ambele derivări aleg aceeași ramură; aplicăm inducția pe acea ramură.
- $c = \text{while } b \text{ do } c_0$: dacă b este fals, ambele derivări folosesc regula While-False și starea finală este σ . Dacă b este adevărat, ambele folosesc While-True. Corpul produce aceeași stare intermediară prin inducție, iar apelul recursiv al buclei din acea stare produce aceeași stare finală prin inducție pe derivarea mai mică.

□

2.2 Dacă un pas small-step este urmat de big-step, atunci big-step-ul original există

Enunț. Dacă $\langle s, \sigma \rangle \rightarrow \langle s', \sigma' \rangle$ și $\langle s', \sigma' \rangle \Downarrow \langle \sigma'' \rangle$, atunci $\langle s, \sigma \rangle \Downarrow \langle \sigma'' \rangle$.

Rezolvare. Facem inducție după regula small-step folosită în primul pas.

Caz atribuire. Dacă

$$\langle x := i, \sigma \rangle \rightarrow \langle \text{skip}, \sigma[x \mapsto i] \rangle,$$

si $\langle \text{skip}, \sigma[x \mapsto i] \rangle \Downarrow \langle \sigma'' \rangle$, atunci prin regula Skip avem $\sigma'' = \sigma[x \mapsto i]$. Cum $\langle i, \sigma \rangle \Downarrow \langle i \rangle$, regula Asgn dă direct

$$\langle x := i, \sigma \rangle \Downarrow \langle \sigma[x \mapsto i] \rangle = \langle \sigma'' \rangle.$$

Caz secvențiere cu primul termen care face pas. Dacă

$$\langle s_1, \sigma \rangle \rightarrow \langle s'_1, \rho \rangle$$

si deci

$$\langle s_1; s_2, \sigma \rangle \rightarrow \langle s'_1; s_2, \rho \rangle,$$

iar $\langle s'_1; s_2, \rho \rangle \Downarrow \langle \sigma'' \rangle$, din regula Seq există τ cu

$$\langle s'_1, \rho \rangle \Downarrow \langle \tau \rangle, \quad \langle s_2, \tau \rangle \Downarrow \langle \sigma'' \rangle.$$

Prin ipoteza de inducție pentru pasul $s_1 \rightarrow s'_1$, rezultă

$$\langle s_1, \sigma \rangle \Downarrow \langle \tau \rangle.$$

Aplicând Seq, obținem $\langle s_1; s_2, \sigma \rangle \Downarrow \langle \sigma'' \rangle$.

Caz secvențiere cu skip. Dacă

$$\langle \text{skip}; s_2, \sigma \rangle \rightarrow \langle s_2, \sigma \rangle,$$

si $\langle s_2, \sigma \rangle \Downarrow \langle \sigma'' \rangle$, atunci prin Skip si Seq avem

$$\langle \text{skip}; s_2, \sigma \rangle \Downarrow \langle \sigma'' \rangle.$$

Caz if. Dacă booleana s-a redus structural, folosim ipoteza de inducție pentru booleană si apoi regula If. Dacă avem axioma

$$\langle \text{if true then } s_1 \text{ else } s_2, \sigma \rangle \rightarrow \langle s_1, \sigma \rangle,$$

atunci din $\langle s_1, \sigma \rangle \Downarrow \langle \sigma'' \rangle$ si din faptul că $\langle \text{true}, \sigma \rangle \Downarrow \langle \text{true} \rangle$ aplicăm If-True. Cazul false este analog.

Caz while. Small-step desfășoară bucla:

$$\text{whilebdo } s \rightarrow \text{ifbthen}(s; \text{whilebdo } s) \text{ else skip}.$$

Dacă forma desfășurată evaluează big-step la σ'' , atunci, prin analiza regulii If:

- dacă b e fals, rezultă $\sigma'' = \sigma$, deci folosim While-False;
- dacă b e adevărat, forma $s; \text{whilebdo } s$ evaluează la σ'' , deci există ρ cu $\langle s, \sigma \rangle \Downarrow \langle \rho \rangle$ si $\langle \text{whilebdo } s, \rho \rangle \Downarrow \langle \sigma'' \rangle$. Aplicăm While-True.

□

2.3 Corespondența small-step / big-step pentru programe terminate

Enunț util. Pentru instrucțiuni IMP terminate, dacă

$$\langle s, \sigma \rangle \rightarrow^* \langle \text{skip}, \sigma' \rangle,$$

atunci

$$\langle s, \sigma \rangle \Downarrow \langle \sigma' \rangle.$$

Rezolvare. Inducție după lungimea secvenței small-step.

Dacă lungimea este 0, atunci configurația inițială este deja $\langle \text{skip}, \sigma' \rangle$, deci $s = \text{skip}$ si $\sigma = \sigma'$. Prin regula Skip, $\langle \text{skip}, \sigma \rangle \Downarrow \langle \sigma' \rangle$.

Pentru pasul inductiv, avem

$$\langle s, \sigma \rangle \rightarrow \langle s_1, \sigma_1 \rangle \rightarrow^* \langle \text{skip}, \sigma' \rangle.$$

Prin ipoteza de inducție aplicată secvenței mai scurte,

$$\langle s_1, \sigma_1 \rangle \Downarrow \langle \sigma' \rangle.$$

Din exercițiul anterior, un pas small-step urmat de evaluare big-step implică evaluarea big-step inițială. Deci

$$\langle s, \sigma \rangle \Downarrow \langle \sigma' \rangle.$$

□

3 C3 - Logică Hoare si cea mai slabă precondiție

3.1 Regula pentru if b then S fără else

Enunț. Cum scriem o regulă Hoare pentru o instrucțiune condițională fără ramură else?

Ideea de examen. O instrucțiune if b then S se comportă ca if b then S else skip. Dacă b este adevărat, executăm S. Dacă b este fals, nu se schimbă starea.

Rezolvare. Regula directă este:

$$\frac{\{P \wedge b\} S \{Q\} \quad \models (P \wedge \neg b) \rightarrow Q}{\{P\} \text{ if } b \text{ then } S \{Q\}}$$

Explicație:

- dacă b este adevărat, suntem în cazul $P \wedge b$ si trebuie ca S să ducă la Q;
- dacă b este fals, instrucțiunea nu face nimic, deci starea finală este aceeași ca starea inițială. Atunci trebuie ca $P \wedge \neg b$ să implice deja Q.

Se poate scrie si ca regulă derivată din regula pentru if-else:

$$\frac{\{P \wedge b\} S \{Q\} \quad \{P \wedge \neg b\} \text{ skip } \{Q\}}{\{P\} \text{ if } b \text{ then } S \text{ else skip } \{Q\}}$$

Dar $\{P \wedge \neg b\} \text{ skip } \{Q\}$ se obține din $\{Q\} \text{ skip } \{Q\}$ prin întărirea precondiției, dacă $P \wedge \neg b \rightarrow Q$.
□

Variantă scurtă de scris.

$$\frac{\{P \wedge b\} S \{Q\} \quad \models P \wedge \neg b \rightarrow Q}{\{P\} \text{ if } b \text{ then } S \{Q\}}$$

Pentru ramura falsă, programul e echivalent cu skip, deci Q trebuie să fie deja adevărat.

3.2 Lema substituției pentru axioma atribuirii

Enunț. În demonstrația corectitudinii regulii de atribuire trebuie arătat: dacă $\sigma \models_I Q[x := e]$, $\langle e, \sigma \rangle \Downarrow \langle n \rangle$ si $\sigma' = \sigma[x \mapsto n]$, atunci $\sigma' \models_I Q$.

Rezolvare. Mai întâi demonstrăm o leamnă pentru expresii aritmetice extinse:

$$|a[x := e]|_{\sigma}^I = |a|_{\sigma[x \mapsto |e|_{\sigma}^I]}^I.$$

Demonstrația este prin inducție structurală după a.

Cazuri:

- $a = N \in \mathbb{Z}$: ambele părți sunt N.
- $a = y$, variabilă de program. Dacă $y = x$, stânga este $|e|_{\sigma}^I$, iar dreapta este $\sigma[x \mapsto |e|_{\sigma}^I](x) = |e|_{\sigma}^I$. Dacă $y \neq x$, stânga este $\sigma(y)$ si dreapta este tot $\sigma(y)$.
- $a = u$, variabilă logică: substituția de program nu o afectează, deci ambele părți sunt $I(u)$.
- $a = a_1 + a_2, a_1 - a_2, a_1 * a_2$: folosim ipoteza de inducție pe subexpresii si compatibilitatea operațiilor.

Acum demonstrăm afirmația pentru asertiunea Q prin inducție structurală.

Caz atomic $a_1 = a_2$. Din $\sigma \models_I (a_1 = a_2)[x := e]$ rezultă

$$|a_1[x := e]|_\sigma^I = |a_2[x := e]|_\sigma^I.$$

Prin lema expresiilor, aceasta este echivalentă cu

$$|a_1|_{\sigma'}^I = |a_2|_{\sigma'}^I,$$

deci $\sigma' \models_I a_1 = a_2$.

Caz atomic $a_1 \leq a_2$. Identic, folosind relația $\leq$.

Cazuri logice. Pentru $\neg A$, $A \wedge B$, $A \vee B$ folosim direct ipotezele de inducție. De exemplu, dacă $Q = A \wedge B$, atunci $Q[x := e] = A[x := e] \wedge B[x := e]$. Din satisfacerea în σ rezultă satisfacerea ambelor componente, iar prin inducție rezultă $\sigma' \models_I A$ și $\sigma' \models_I B$.

Caz cuantificator $\forall y.A$. Dacă variabila logică y este diferită de variabilele de program, substituția $[x := e]$ nu capturează nimic. Avem:

$$(\forall y.A)[x := e] = \forall y.(A[x := e]).$$

Presupunem $\sigma \models_I \forall y.(A[x := e])$. Pentru orice $N \in \mathbb{Z}$, avem

$$\sigma \models_{I[y \mapsto N]} A[x := e].$$

Aplicând ipoteza de inducție pentru A și interpretarea $I[y \mapsto N]$, obținem

$$\sigma' \models_{I[y \mapsto N]} A.$$

Cum N a fost arbitrar, $\sigma' \models_I \forall y.A$.

Astfel, $\sigma \models_I Q[x := e]$ implică $\sigma[x \mapsto |e|_\sigma^I] \models_I Q$. Cum $\langle e, \sigma \rangle \Downarrow \langle n \rangle$ implică $|e|_\sigma^I = n$, rezultă $\sigma' \models_I Q$. $\square$

3.3 Echivalența dintre triplet Hoare semantic și wls

Enunț. Arătați că pentru orice A, B, c, I :

$$\models_I \{A\}c\{B\} \quad \text{dacă și numai dacă} \quad A^I \subseteq wls_I(c, B).$$

Rezolvare. Prin definiție,

$$A^I = \{\sigma \in \Sigma \mid \sigma \models_I A\}.$$

De asemenea,

$$wls_I(c, B) = \{\sigma \in \Sigma \mid \forall \sigma' (\langle c, \sigma \rangle \Downarrow \langle \sigma' \rangle \Rightarrow \sigma' \models_I B)\}.$$

($\Rightarrow$) Presupunem $\models_I \{A\}c\{B\}$. Fie $\sigma \in A^I$. Atunci $\sigma \models_I A$. Trebuie să arătăm că $\sigma \in wls_I(c, B)$. Fie orice σ' cu $\langle c, \sigma \rangle \Downarrow \langle \sigma' \rangle$. Din semantica tripletului Hoare, deoarece precondiția A era adevărată înainte, rezultă $\sigma' \models_I B$. Deci $\sigma \in wls_I(c, B)$.

($\Leftarrow$) Presupunem $A^I \subseteq wls_I(c, B)$. Trebuie să arătăm $\models_I \{A\}c\{B\}$. Fie σ cu $\sigma \models_I A$ și fie σ' cu $\langle c, \sigma \rangle \Downarrow \langle \sigma' \rangle$. Din $\sigma \models_I A$ avem $\sigma \in A^I$, deci prin incluziune $\sigma \in wls_I(c, B)$. Prin definiția lui wls , orice stare finală obținută din σ satisface B , deci $\sigma' \models_I B$. $\square$

3.4 Când execuția termină mereu, $ws = wls$

Enunț. Dacă execuția lui c se termină în orice stare, atunci $ws_I(c, B) = wls_I(c, B)$.

Rezolvare. Definițiile sunt:

$$ws_I(c, B) = \{\sigma \mid \exists \sigma' (\langle c, \sigma \rangle \Downarrow \langle \sigma' \rangle \wedge \sigma' \models_I B)\},$$

$$wls_I(c, B) = \{\sigma \mid \forall \sigma' (\langle c, \sigma \rangle \Downarrow \langle \sigma' \rangle \Rightarrow \sigma' \models_I B)\}.$$

În general, pentru limbaje nedeterministe, $\exists$ si $\forall$ pot fi diferite. Pentru IMP, semantica big-step este deterministă: dintr-o stare inițială, un program terminat are cel mult o stare finală.

Incluziunea $wls \subseteq ws$. Fie $\sigma \in wls_I(c, B)$. Prin ipoteza de terminare totală, există σ' cu $\langle c, \sigma \rangle \Downarrow \langle \sigma' \rangle$. Cum $\sigma \in wls$, orice stare finală satisface B , deci $\sigma' \models_I B$. Prin urmare $\sigma \in ws_I(c, B)$.

Incluziunea $ws \subseteq wls$. Fie $\sigma \in ws_I(c, B)$. Există ρ cu

$$\langle c, \sigma \rangle \Downarrow \langle \rho \rangle, \quad \rho \models_I B.$$

Trebuie să arătăm că orice σ' cu $\langle c, \sigma \rangle \Downarrow \langle \sigma' \rangle$ satisface B . Dar prin determinismul big-step, $\sigma' = \rho$. Deci $\sigma' \models_I B$.

Rezultă egalitatea. □

3.5 Exercițiul din demonstrația $wp(\text{while}_b S, B)^I = ws_I(\text{while}_b S, B)$

Enunț. În demonstrația direcției $ws \subseteq wp$, se folosește lema: dacă execuția buclei are n iterații si stările sunt $\sigma_0, \dots, \sigma_n$, atunci pentru orice $j \leq n$, avem

$$\sigma_{n-j} \models_I P_j,$$

unde

$$P_0 := \neg b \wedge B, \quad P_{k+1} := b \wedge wp(S, P_k).$$

Demonstrați prin inducție după j .

Rezolvare. Presupunem că

$$\sigma_0 = \sigma, \quad \sigma_n = \sigma', \quad \langle b, \sigma_n \rangle \Downarrow \langle \text{false} \rangle,$$

si pentru fiecare $0 \leq i < n$:

$$\langle b, \sigma_i \rangle \Downarrow \langle \text{true} \rangle, \quad \langle S, \sigma_i \rangle \Downarrow \langle \sigma_{i+1} \rangle.$$

Mai presupunem că starea finală satisface postcondiția:

$$\sigma_n \models_I B.$$

Baza $j = 0$. Trebuie să arătăm $\sigma_n \models_I P_0$. Dar $P_0 = \neg b \wedge B$. Avem $\langle b, \sigma_n \rangle \Downarrow \langle \text{false} \rangle$, deci $\sigma_n \models_I \neg b$, si avem deja $\sigma_n \models_I B$. Deci $\sigma_n \models_I P_0$.

Pas inductiv. Presupunem că $\sigma_{n-j} \models_I P_j$. Trebuie să arătăm

$$\sigma_{n-(j+1)} \models_I P_{j+1}.$$

Prin definiție,

$$P_{j+1} = b \wedge wp(S, P_j).$$

Deci trebuie două lucruri:

1. $\sigma_{n-j-1} \models_I b$;
2. $\sigma_{n-j-1} \models_I \text{wp}(S, P_j)$.

Primul rezultă pentru că $n - j - 1 < n$, deci condiția buclei era adevărată în acea stare:

$$\langle b, \sigma_{n-j-1} \rangle \Downarrow \langle \text{true} \rangle.$$

Pentru al doilea, știm că

$$\langle S, \sigma_{n-j-1} \rangle \Downarrow \langle \sigma_{n-j} \rangle$$

si, prin ipoteza de inducție, $\sigma_{n-j} \models_I P_j$. Deci $\sigma_{n-j-1} \in \text{ws}_I(S, P_j)$. Prin ipoteza de inducție a teoremei pentru instrucțiunea S ,

$$\text{wp}(S, P_j)^I = \text{ws}_I(S, P_j),$$

rezultă $\sigma_{n-j-1} \models_I \text{wp}(S, P_j)$. Prin urmare

$$\sigma_{n-j-1} \models_I b \wedge \text{wp}(S, P_j) = P_{j+1}.$$

Inducția este completă. □

4 C4 - Unificare

4.1 Lema ajutătoare pentru regula REZOLVĂ

Enunț. Fie Θ unificator pentru ecuația $x = t$, unde x este variabilă. Dacă $x \notin \text{Var}(t)$, atunci

$$\Theta = \tilde{\Theta} \circ [x \mapsto t].$$

Rezolvare. Trebuie să arătăm că cele două substituții dau aceeași valoare pe orice variabilă y .

Dacă $y = x$, atunci

$$(\tilde{\Theta} \circ [x \mapsto t])(x) = \tilde{\Theta}(t).$$

Cum Θ este unificator pentru $x = t$, avem

$$\tilde{\Theta}(x) = \tilde{\Theta}(t).$$

Dar $\tilde{\Theta}(x) = \Theta(x)$. Deci

$$(\tilde{\Theta} \circ [x \mapsto t])(x) = \Theta(x).$$

Dacă $y \neq x$, atunci $[x \mapsto t](y) = y$, deci

$$(\tilde{\Theta} \circ [x \mapsto t])(y) = \tilde{\Theta}(y) = \Theta(y).$$

Cele două substituții coincid pe toate variabilele, deci sunt egale. □

4.2 Corectitudinea invariantului pentru SCOATE

Enunț. Pentru pasul

$$(\Theta, R \cup \{t = t\}) \Rightarrow (\Theta, R),$$

arătați că invariantul

$$U(E) = \{\Theta' \in U(R) \mid \Theta \succ \Theta'\}$$

se păstrează.

Rezolvare. Ecuația $t = t$ este satisfăcută de orice substituție. Deci

$$U(R \cup \{t = t\}) = U(R).$$

Prima componentă Θ nu se schimbă. Prin urmare mulțimea

$$\{\Theta' \in U(R \cup \{t = t\}) \mid \Theta \succ \Theta'\}$$

este identică cu

$$\{\Theta' \in U(R) \mid \Theta \succ \Theta'\}.$$

Invariantul este păstrat. □

4.3 Corectitudinea invariantului pentru DESCOMPUNE

Enunț. Pentru pasul

$$f(t_1, \dots, t_n) = f(t'_1, \dots, t'_n)$$

înlocuit cu ecuațiile

$$t_1 = t'_1, \dots, t_n = t'_n,$$

arătați că unificatorii nu se schimbă.

Rezolvare. Pentru orice substituție Δ ,

$$\Delta \in U(\{f(t_1, \dots, t_n) = f(t'_1, \dots, t'_n)\})$$

înseamnă

$$\tilde{\Delta}(f(t_1, \dots, t_n)) = \tilde{\Delta}(f(t'_1, \dots, t'_n)).$$

Prin definiția extinderii substituției,

$$f(\tilde{\Delta}(t_1), \dots, \tilde{\Delta}(t_n)) = f(\tilde{\Delta}(t'_1), \dots, \tilde{\Delta}(t'_n)).$$

Termenii cu același simbol de funcție f sunt egali exact când argumentele corespunzătoare sunt egale. Deci

$$\tilde{\Delta}(t_i) = \tilde{\Delta}(t'_i) \quad \text{pentru orice } i.$$

Aceasta este echivalent cu Δ unifică toate ecuațiile $t_i = t'_i$. Deci DESCOMPUNE păstrează exact aceleași mulțime de unificatori. Cum Θ nu se schimbă, se păstrează și invariantul. $\square$

4.4 Corectitudinea invariantului pentru REZOLVĂ

Enunț. Pentru ecuația $x = t$, cu $x \notin \text{Var}(t)$, algoritmul trece de la

$$(\Theta, R \cup \{x = t\})$$

la

$$([x \mapsto t] \circ \Theta, [x \mapsto t](R)).$$

Explicați de ce invariantul se păstrează.

Rezolvare. Notăm $\eta = [x \mapsto t]$. Noua substituție acumulată este $\eta \circ \Theta$, iar noul sistem de ecuații este $\eta(R)$.

Un unificator al vechiului sistem trebuie să unifice și $x = t$. Prin lema precedentă, orice astfel de unificator Δ se poate scrie

$$\Delta = \tilde{\Delta} \circ \eta.$$

În plus, Δ unifică ecuațiile din R dacă și numai dacă partea rămasă unifică ecuațiile din $\eta(R)$. Intuitiv, după ce am decis că x trebuie înlocuit cu t , aplicăm această decizie peste toate ecuațiile rămase.

Mai formal, pentru orice ecuație $u = v \in R$:

$$\tilde{\Delta}(u) = \tilde{\Delta}(v)$$

este echivalent, folosind $\Delta = \tilde{\Delta} \circ \eta$, cu

$$\tilde{\Delta}(\eta(u)) = \tilde{\Delta}(\eta(v)).$$

Deci unificatorii vechiului sistem sunt exact substituțiile obținute din unificatorii sistemului nou prin compunerea cu η .

Aceasta este exact actualizarea făcută de algoritm: mută informația $x \mapsto t$ din mulțimea de ecuații în substituția acumulată. Prin urmare invariantul se păstrează. $\square$

4.5 Varianta algoritmului de unificare și terminarea

Enunț. Se definește

$$\text{variant}(\Theta, R) = (|\text{Var}(R)|, n_F(R), |R|)$$

cu ordine lexicografică. Arătați că fiecare pas scade varianta și deduceți terminarea.

Rezolvare. Analizăm regulile.

SCOATE. Se elimină ecuația $t = t$. Nu apar variabile sau simboluri funcționale noi. Primele două componente nu cresc, iar $|R|$ scade cu 1. Deci tripletul scade lexicografic.

DESCOMPUNE. Ecuația

$$f(t_1, \dots, t_n) = f(t'_1, \dots, t'_n)$$

este înlocuită cu n ecuații între argumente. Mulțimea variabilelor nu crește. Numărul de simboluri funcționale scade, deoarece dispar cele două apariții exterioare ale lui f . Chiar dacă numărul de ecuații poate crește, componenta a doua scade, deci varianta scade lexicografic.

REZOLVĂ. Ecuația $x = t$ se rezolvă cu $x \notin \text{Var}(t)$. Variabila x dispăre din toate ecuațiile rămase, fiind înlocuită cu t , iar t nu conține x . Nu apar variabile noi. Deci $|\text{Var}(R)|$ scade. Prin urmare varianta scade lexicografic.

Cum ordinea lexicografică pe $\mathbb{N}^3$ este bine fondată, nu există șir infinit strict descrescător. Dacă algoritmul ar admite o derivare infinită, variantele configurațiilor ar forma un astfel de șir infinit descrescător, contradicție. Deci algoritmul termină. $\square$

4.6 Corectitudinea parțială și totală a algoritmului de unificare

Enunț. Dacă algoritmul ajunge într-o configurație finală (Θ, R) , atunci:

- dacă $R = \emptyset$, Θ este cel mai general unificator pentru E ;
- dacă $R \neq \emptyset$, atunci E nu are unificatori.

Rezolvare. Folosim invariantul:

$$U(E) = \{\Theta' \in U(R) \mid \Theta \succ \Theta'\}.$$

Cazul $R = \emptyset$. Orice substituție unifică sistemul vid de ecuații, deci

$$U(R) = T_\sigma^V,$$

adică mulțimea tuturor substituțiilor. Atunci invariantul devine

$$U(E) = \{\Theta' \mid \Theta \succ \Theta'\}.$$

Aceasta este exact definiția alternativă a faptului că Θ este cel mai general unificator al lui E .

Cazul $R \neq \emptyset$ **final.** O configurație finală nenulă conține doar eșecuri:

- conflict de simboluri: $f(\dots) = g(\dots)$ cu $f \neq g$;
- ciclu: $x = t$ cu $x \in \text{Var}(t)$ și $t \neq x$.

În ambele situații, $U(R) = \emptyset$. Pentru conflict, nicio substituție nu poate transforma doi termeni cu simboluri externe diferite în termeni egali. Pentru ciclu, ar trebui ca un

termen să fie egal cu un termen care îl conține strict, ceea ce ar produce un termen infinit; în algebra termenilor fini este imposibil. Deci, prin invariant,

$$U(E) = \{\Theta' \in \mathcal{O} \mid \Theta \succ \Theta'\} = \emptyset.$$

Terminarea a fost demonstrată anterior. Prin urmare algoritmul este total corect: se oprește si răspunde corect, fie cu mgu, fie cu eșec. $\square$

5 C5 - Prolog, rezoluție, semantică

5.1 Termeni, formule atomice și enunțuri pentru alfabetul dat

Enunț. Fie alfabetul definit prin $R = \{<\}$, $F = \{s, +, 0\}$ și aritățile $r(0) = 0$, $r(s) = 1$, $r(+) = r(<) = 2$. Dați 3 termeni, 3 formule atomice și 3 enunțuri.

Rezolvare. Simbolurile funcționale sunt 0, s , $+$. Deci termenii se construiesc din variabile, constanta 0, succesorul s și operația binară $+$.

Exemple de 3 termeni:

$$0, \quad s(0), \quad +(x, s(0)).$$

Putem scrie ultimul și infixat: $x + s(0)$.

Singurul simbol relațional este $<$, de aritate 2. Deci formulele atomice relaționale au forma $<(t_1, t_2)$, unde t_1, t_2 sunt termeni.

Exemple de 3 formule atomice:

$$<(0, s(0)), \quad <(x, s(x)), \quad <(+ (x, 0), + (s(0), y)).$$

Un enunț este o formulă fără variabile libere, deci cuantificăm toate variabilele.

Exemple de 3 enunțuri:

$$\forall x \quad <(x, s(x)),$$

$$\forall x \forall y \quad <(x, + (x, s(y))),$$

$$\forall x \quad (\neg <(s(x), x)).$$

Al treilea folosește conectorul derivat $\neg$; este acceptabil ca formulă de ordinul I. $\square$

5.2 Arborele SLD pentru ?- $p(X, X)$

Enunț. Pentru programul:

1. $p(X, Y) :- q(X, Z), r(Z, Y).$
2. $p(X, X) :- s(X).$
3. $q(X, b).$
4. $q(b, a).$
5. $q(X, a) :- r(a, X).$
6. $r(b, a).$
7. $s(X) :- t(X, a).$
8. $s(X) :- t(X, b).$
9. $s(X) :- t(X, X).$
10. $t(a, b).$
11. $t(b, a).$

Desenați arborele SLD pentru scopul ?- $p(X, X)$.

Rezolvare. Scopul inițial este

$$G_0 = \neg p(X, X).$$

Prolog/SLD încearcă regulile pentru p .

Ramura cu regula 1. Regula 1, cu variabile redenumite:

$$p(U, V) \leftarrow q(U, Z_1), r(Z_1, V).$$

Unificăm $p(X, X)$ cu $p(U, V)$, deci $U = X, V = X$. Obținem scopul:

$$\neg q(X, Z_1) \vee \neg r(Z_1, X).$$

Rezolvăm primul subscop $q(X, Z_1)$.

Regula 3: $q(U, b)$. Unificare cu $q(X, Z_1)$ dă $Z_1 = b$. Scop nou:

$$\neg r(b, X).$$

Regula 6: $r(b, a)$. Unificare dă $X = a$. Scopul devine $\square$. Soluție: $X = a$.

Regula 4: $q(b, a)$. Unificare cu $q(X, Z_1)$ dă $X = b, Z_1 = a$. Scop nou:

$$\neg r(a, b).$$

Nu există fapt/regulă pentru $r(a, b)$; ramura eșuează.

Regula 5: $q(U, a) \leftarrow r(a, U)$. Unificare cu $q(X, Z_1)$ dă $Z_1 = a, U = X$. Scop nou:

$$\neg r(a, X) \vee \neg r(a, X).$$

Nu există $r(a, \cdot)$; eșec.

Regula 6 nu este pentru q , deci nu se aplică. Totuși în desenul cursului apare și încercarea lui $r(Z_1, X)$ după anumite unificări; ramura de succes relevantă rămâne $X = a$.

Ramura cu regula 2. Regula 2:

$$p(U, U) \leftarrow s(U).$$

Unificarea lui $p(X, X)$ cu $p(U, U)$ dă $U = X$. Scopul devine

$$\neg s(X).$$

Aplicăm regulile pentru s .

Regula 7: $s(U) \leftarrow t(U, a)$. Scop:

$$\neg t(X, a).$$

Faptul 11 este $t(b, a)$, deci $X = b$ și obținem $\square$. Soluție: $X = b$. Faptul 10, $t(a, b)$, nu unifică cu $t(X, a)$, deoarece al doilea argument ar trebui să fie simultan b și a .

Regula 8: $s(U) \leftarrow t(U, b)$. Scop:

$$\neg t(X, b).$$

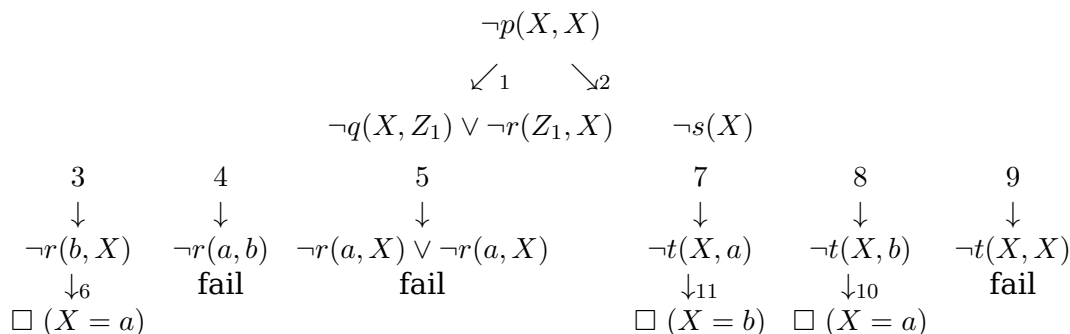
Faptul 10 este $t(a, b)$, deci $X = a$ și obținem $\square$. Soluție: $X = a$. Faptul 11 nu se potrivește.

Regula 9: $s(U) \leftarrow t(U, U)$. Scop:

$$\neg t(X, X).$$

Faptul 10 $t(a, b)$ nu unifică, fiindcă ar cere $X = a$ și $X = b$. Faptul 11 $t(b, a)$ nu unifică, fiindcă ar cere $X = b$ și $X = a$. Ramura eșuează.

Arbore schematic:



Soluțiile găsite sunt $X = a$ și $X = b$; $X = a$ apare pe două ramuri diferite. $\square$

5.3 Proprietăți ale satisfacției pentru mulțimi de enunțuri

Enunț. Fie $\Gamma \subseteq E_\sigma$, $\Delta \subseteq \Gamma$ și $\mathcal{A}$ o structură. Demonstrați:

- (a) dacă $\mathcal{A} \models \Gamma$, atunci $\mathcal{A} \models \Delta$;
- (b) dacă Δ este nesatisfiabilă, atunci Γ este nesatisfiabilă;
- (c) $\mathcal{A} \models \Gamma$ dacă și numai dacă pentru orice $\Sigma \subseteq \Gamma$ finită, $\mathcal{A} \models \Sigma$.

Rezolvare. (a) Prin definiție, $\mathcal{A} \models \Gamma$ înseamnă că pentru orice $\varphi \in \Gamma$, $\mathcal{A} \models \varphi$. Cum $\Delta \subseteq \Gamma$, orice $\varphi \in \Delta$ este și în Γ , deci $\mathcal{A} \models \varphi$. Prin urmare $\mathcal{A} \models \Delta$.

(b) Presupunem prin reducere la absurd că Γ este satisfiabilă. Atunci există $\mathcal{A}$ cu $\mathcal{A} \models \Gamma$. Prin (a), pentru că $\Delta \subseteq \Gamma$, avem $\mathcal{A} \models \Delta$. Asta contrazice nesatisfiabilitatea lui Δ . Deci Γ este nesatisfiabilă.

(c) Dacă $\mathcal{A} \models \Gamma$, atunci prin (a) $\mathcal{A}$ satisface orice submulțime finită $\Sigma \subseteq \Gamma$.

Invers, presupunem că $\mathcal{A} \models \Sigma$ pentru orice $\Sigma \subseteq \Gamma$ finită. Fie $\varphi \in \Gamma$. Luăm $\Sigma = \{\varphi\}$, care este finită și inclusă în Γ . Atunci $\mathcal{A} \models \Sigma$, deci $\mathcal{A} \models \varphi$. Cum φ a fost arbitrar, $\mathcal{A} \models \Gamma$. $\square$

5.4 Proprietăți ale deducției semantice

Enunț. Fie $\Gamma \subseteq E_\sigma$, $\Delta \subseteq \Gamma$ și $\varphi \in E_\sigma$. Demonstrați:

- (a) dacă $\Delta \models \varphi$, atunci $\Gamma \models \varphi$;
- (b) Γ este nesatisfiabilă dacă și numai dacă $\Gamma \models \perp$;
- (c) $\Gamma \models \varphi$ dacă și numai dacă $\Gamma \cup \{\neg\varphi\}$ este nesatisfiabilă.

Rezolvare. (a) Presupunem $\Delta \models \varphi$. Fie o structură $\mathcal{A}$ cu $\mathcal{A} \models \Gamma$. Deoarece $\Delta \subseteq \Gamma$, rezultă $\mathcal{A} \models \Delta$. Din $\Delta \models \varphi$, rezultă $\mathcal{A} \models \varphi$. Deci orice model al lui Γ este model al lui φ , adică $\Gamma \models \varphi$.

(b) Dacă Γ este nesatisfiabilă, atunci nu există niciun model al lui Γ . Afirmatia “orice model al lui Γ satisface $\perp$ ” este adevărată vid. Deci $\Gamma \models \perp$.

Invers, dacă $\Gamma \models \perp$ și ar exista $\mathcal{A}$ cu $\mathcal{A} \models \Gamma$, atunci ar trebui $\mathcal{A} \models \perp$. Dar $\perp$ nu este satisfăcută în nicio structură. Contradicție. Deci Γ este nesatisfiabilă.

(c) Presupunem $\Gamma \models \varphi$. Dacă $\Gamma \cup \{\neg\varphi\}$ ar avea un model $\mathcal{A}$, atunci $\mathcal{A} \models \Gamma$ și $\mathcal{A} \models \neg\varphi$. Din $\Gamma \models \varphi$, am avea $\mathcal{A} \models \varphi$, contradicție. Deci $\Gamma \cup \{\neg\varphi\}$ este nesatisfiabilă.

Invers, presupunem că $\Gamma \cup \{\neg\varphi\}$ este nesatisfiabilă. Fie $\mathcal{A}$ cu $\mathcal{A} \models \Gamma$. Dacă $\mathcal{A} \not\models \varphi$, atunci $\mathcal{A} \models \neg\varphi$, deci $\mathcal{A} \models \Gamma \cup \{\neg\varphi\}$, contradicție. Prin urmare $\mathcal{A} \models \varphi$. Deci $\Gamma \models \varphi$. $\square$

5.5 Proprietățile substituției în logica de ordinul I

Enunț. Fie θ o substituție și $\mathcal{A}$ o structură. Pentru o valoare v , definim v' prin

$$v'(x) = (\theta(x))_v^{\mathcal{A}}.$$

Demonstrați:

- (a) pentru orice termen t , $(t\theta)_v^{\mathcal{A}} = t_{v'}^{\mathcal{A}}$;
- (b) pentru orice formulă φ , $\mathcal{A} \models_{v'} \varphi$ dacă și numai dacă $\mathcal{A} \models_v \varphi\theta$.

Rezolvare. (a) Termeni. Inducție structurală după t .

Dacă $t = x$ este variabilă, atunci

$$(t\theta)_v^A = (\theta(x))_v^A = v'(x) = x_{v'}^A.$$

Dacă $t = f(t_1, \dots, t_n)$, atunci

$$(t\theta)_v^A = (f(t_1\theta, \dots, t_n\theta))_v^A = f^A((t_1\theta)_v^A, \dots, (t_n\theta)_v^A).$$

Prin ipoteza de inducție,

$$(t_i\theta)_v^A = (t_i)_{v'}^A.$$

Deci

$$(t\theta)_v^A = f^A((t_1)_{v'}^A, \dots, (t_n)_{v'}^A) = t_{v'}^A.$$

(b) Formule. Inducție structurală după φ .

Dacă φ este ecuația $t = u$, atunci

$$\mathcal{A} \models_v (t = u)\theta \iff (t\theta)_v^A = (u\theta)_v^A.$$

Prin partea (a), aceasta este echivalentă cu

$$t_{v'}^A = u_{v'}^A,$$

adică $\mathcal{A} \models_{v'} t = u$.

Dacă φ este relațională $p(t_1, \dots, t_n)$, folosim partea (a) pentru fiecare argument și definiția interpretării relației p^A .

Cazurile pentru $\perp$, $\rightarrow$, conectori derivați se tratează direct din definiția satisfacției și ipoteza de inducție.

Pentru $\forall x\psi$, trebuie avută grijă la variabile legate. În forma standard a substituției se presupune că substituția este aplicată doar variabilelor libere, cu redenumire dacă este nevoie, pentru a evita capturarea. Atunci:

$$\mathcal{A} \models_{v'} \forall x\psi \iff \text{pentru orice } a \in A, \mathcal{A} \models_{v'[x \leftarrow a]} \psi.$$

Prin ipoteza de inducție, aceasta este echivalentă cu

$$\text{pentru orice } a \in A, \mathcal{A} \models_{v[x \leftarrow a]} \psi\theta,$$

adică

$$\mathcal{A} \models_v \forall x(\psi\theta) = (\forall x\psi)\theta.$$

Deci proprietatea este adevărată. □

5.6 De ce $B_\sigma \models_H P$ rămâne adevărat

Enunț. În demonstrația că $K_P = \{J \subseteq B_\sigma \mid J \models_H P\}$ este mulțime Moore pe B_σ , se afirmă că $B_\sigma \models_H P$. Demonstrați.

Rezolvare. B_σ este mulțimea tuturor atomilor de bază, adică atomilor fără variabile. Relația $J \models_H P$ pentru o clauză definită spune: pentru orice instanță de bază

$$A_0 \leftarrow A_1, \dots, A_m,$$

dacă $A_1, \dots, A_m \in J$, atunci $A_0 \in J$.

Luăm $J = B_\sigma$. Trebuie să arătăm că orice clauză din P este satisfăcută Herbrand de B_σ .

Fie o instanță de bază a unei clauze:

$$A_0 \leftarrow A_1, \dots, A_m.$$

Presupunem că $A_i \in B_\sigma$ pentru toate $i \geq 1$. Trebuie să arătăm $A_0 \in B_\sigma$.

Dar A_0 este tot atom de bază, fiind capul aceleiași instanțe de bază. Prin definiția lui B_σ , orice atom de bază aparține lui B_σ . Deci $A_0 \in B_\sigma$.

Prin urmare fiecare clauză din P este satisfăcută de B_σ , deci $B_\sigma \models_H P$. $\square$

5.7 Corectitudinea rezoluției SLD - demonstrație de examen

Enunț. Dacă θ este o soluție calculată de o respingere SLD pentru scopul $A_1, \dots, A_m$, atunci programul P implică semantic răspunsul calculat:

$$P \models \forall(A_1 \wedge \dots \wedge A_m)\theta.$$

Rezolvare. Demonstrația este prin inducție după lungimea respingerii SLD.

Caz de bază. Respingerea are un singur pas. Atunci scopul este un singur atom A_1 , iar clauza folosită este un fapt B_0 . Unificatorul θ face $A_1\theta = B_0\theta$. Cum B_0 vine din programul P , orice model al lui P satisface B_0 , deci satisface $B_0\theta$, deci satisface $A_1\theta$.

Pas inductiv. Presupunem că primul pas rezolvă atomul A_i folosind clauza redenumită

$$B_0 \leftarrow B_1, \dots, B_k$$

si cgu-ul θ_0 pentru A_i si B_0 . Noul scop este

$$A_1, \dots, A_{i-1}, B_1, \dots, B_k, A_{i+1}, \dots, A_m$$

apoi se continuă cu o respingere mai scurtă si substituția totală θ .

Prin ipoteza de inducție pentru respingerea mai scurtă, orice model al lui P satisface

$$(A_1 \wedge \dots \wedge A_{i-1} \wedge B_1 \wedge \dots \wedge B_k \wedge A_{i+1} \wedge \dots \wedge A_m)\theta.$$

Pe de altă parte, clauza folosită este în P , deci orice model al lui P satisface

$$(B_1 \wedge \dots \wedge B_k)\theta \rightarrow B_0\theta.$$

Cum θ extinde cgu-ul lui A_i si B_0 , avem

$$B_0\theta = A_i\theta.$$

Rezultă că orice model al lui P satisface si $A_i\theta$. Împreună cu atomii deja obținuți, avem

$$P \models \forall(A_1 \wedge \dots \wedge A_m)\theta.$$

$\square$

6 C6 - Lambda-calcul

6.1 Eliminarea parantezelor inutile

Enunț. Scrieți termenii cu cât mai puține paranteze, fără a schimba sensul:

(a) $(\lambda x.(\lambda y.(\lambda z.((xz)(yz)))))$;

(b) $((ab)(cd))((ef)(gh))$.

Rezolvare. Reguli folosite:

- aplicația este asociativă la stânga: MNP înseamnă $(MN)P$;
- corpul lui λ se extinde cât mai mult la dreapta;
- $\lambda xyz.M$ abreviază $\lambda x.\lambda y.\lambda z.M$.

(a) Termenul

$$(\lambda x.(\lambda y.(\lambda z.((xz)(yz)))))$$

se scrie

$$\lambda xyz. xz(yz).$$

Aici $xz(yz)$ înseamnă $(xz)(yz)$, nu $x(z(yz))$, deoarece aplicația este asociativă la stânga doar pentru lanțuri fără paranteze, iar yz trebuie păstrat ca argument compus.

(b) Termenul

$$(((ab)(cd))((ef)(gh)))$$

se scrie

$$ab(cd)(ef(gh))$$

cu atenție: pentru a păstra exact structura, forma clară minimală este

$$(ab)(cd)((ef)(gh)).$$

Dacă scriem $ab(cd)(ef)(gh)$, obținem $((ab)(cd))(ef)(gh)$, care este alt termen. Deci păstrăm parantezele din $(ef)(gh)$. $\square$

Variantă scurtă de scris. (a) $\lambda xyz. xz(yz)$.

(b) $(ab)(cd)((ef)(gh))$.

6.2 Adăugarea parantezelor după convenții

Enunț. Adăugați paranteze fără a schimba sensul:

(a) $x x x x$;

(b) $\lambda x. x \lambda y. y$.

Rezolvare. (a) Aplicația este asociativă la stânga:

$$x x x x = (((xx)x)x).$$

(b) Corpul lui λx . se extinde cât mai mult la dreapta, deci

$$\lambda x. x \lambda y. y = \lambda x. (x(\lambda y. y)).$$

Aici $\lambda y. y$ este argumentul lui x . $\square$

6.3 Substituții lambda - exemplul 1

Enunț. Calculați

$$(\lambda y.(x(\lambda w.((vw)x))))[x := uv].$$

Variabilele u, v, w, x, y, z sunt distincte.

Rezolvare. În termenul

$$\lambda y.(x(\lambda w.((vw)x))),$$

variabila legată este y . Substituim x cu uv . Deoarece $y \notin FV(uv) = \{u, v\}$, nu trebuie redenumit y .

Intrăm în corp:

$$x(\lambda w.((vw)x)).$$

Prima apariție x devine uv .

Pentru $\lambda w.((vw)x)$, avem $w \notin FV(uv) = \{u, v\}$, deci nu redenumim w și substituim în corp:

$$((vw)x)[x := uv] = (vw)(uv).$$

Deci rezultatul este

$$\lambda y.((uv)(\lambda w.((vw)(uv)))).$$

Cu paranteze reduse:

$$\lambda y. (uv)(\lambda w. vw(uv)).$$

□

6.4 Substituții lambda - exemplul 2, cu redenumire pentru evitarea capturării

Enunț. Calculați

$$(\lambda y.(x(\lambda x.x)))[x := \lambda y.(xy)].$$

Rezolvare. Termenul substituit este

$$N = \lambda y.(xy).$$

Variabilele libere ale lui N sunt $FV(N) = \{x\}$, deoarece y este legată.

Avem termenul exterior $\lambda y.(x(\lambda x.x))$. Substituim pentru x . Binderul exterior este y , iar $y \notin FV(N)$, deci nu capturează variabile libere din N . Putem intra în corp:

$$(x(\lambda x.x))[x := N].$$

Prima componentă x devine N . Pentru $\lambda x.x$, binderul este chiar variabila substituită, deci substituția se oprește:

$$(\lambda x.x)[x := N] = \lambda x.x.$$

Rezultatul este

$$\lambda y.((\lambda y.(xy))(\lambda x.x)).$$

Observație: nu este capturată nicio apariție liberă, fiindcă N nu are y liber; y din N este legat intern. □

6.5 Substituții lambda - exemplul 3, captură evitată

Enunț. Calculați

$$(y(\lambda v.(xv)))[x := \lambda y.(vy)].$$

Rezolvare. Termenul substituit este

$$N = \lambda y.(vy).$$

Variabilele libere ale lui N sunt $FV(N) = \{v\}$, deoarece y este legată.

Substituim în aplicație:

$$(y(\lambda v.(xv)))[x := N] = y[x := N] ((\lambda v.(xv))[x := N]).$$

Cum $y \neq x$, $y[x := N] = y$.

Acum analizăm $\lambda v.(xv)$. Binderul este v , iar $v \in FV(N)$. Dacă am intra direct, v liber din N ar fi capturat de λv . Deci trebuie să redenumim variabila legată v cu o variabilă nouă, de exemplu z :

$$\lambda v.(xv) \equiv_{\alpha} \lambda z.(xz).$$

Acum substituim:

$$(\lambda z.(xz))[x := N] = \lambda z.((xz)[x := N]) = \lambda z.(Nz).$$

Înlocuind N :

$$\lambda z.((\lambda y.(vy))z).$$

Rezultatul final este

$$y(\lambda z.((\lambda y.(vy))z)).$$

□

6.6 Substituții lambda - exemplul 4

Enunț. Calculați

$$(\lambda x.(zy))[x := uv].$$

Rezolvare. Termenul are forma $\lambda x.P$, iar substituția este chiar pentru variabila x . Regula spune:

$$(\lambda x.P)[x := N] = \lambda x.P.$$

Motivul: aparițiile lui x din corpul lui $\lambda x.P$ sunt legate de acel λ , deci nu sunt apariții libere care trebuie înlocuite.

Prin urmare

$$(\lambda x.(zy))[x := uv] = \lambda x.(zy).$$

□

6.7 Termeni care pot fi aduși la formă normală

Enunț. Pot fi termenii de mai jos aduși la o β -formă normală?

- (a) $(\lambda x.x)M$;
- (b) $(\lambda xy.x)MN$;
- (c) $(\lambda x.xx)(\lambda y.yyy)$.

Rezolvare. (a)

$$(\lambda x.x)M \rightarrow_{\beta} x[x := M] = M.$$

Deci termenul are formă normală dacă M are formă normală. Mai precis, termenul poate fi adus la o formă normală exact când M poate fi adus la o formă normală.

(b) Amintim că

$$\lambda xy.x = \lambda x.(\lambda y.x).$$

Atunci

$$(\lambda xy.x)MN = ((\lambda x.\lambda y.x)M)N \rightarrow_{\beta} (\lambda y.M)N \rightarrow_{\beta} M.$$

Deci, la fel, termenul poate fi adus la formă normală exact când M poate fi adus la formă normală. Observație: N este ignorat.

(c) Notăm

$$A = \lambda y.yyy.$$

Termenul este

$$(\lambda x.xx)A \rightarrow_{\beta} AA.$$

Acum

$$AA = (\lambda y.yyy)A \rightarrow_{\beta} AAA.$$

Acest termen conține iar AA ca prefix si produce tot mai multe copii ale lui A :

$$AA \rightarrow_{\beta} AAA \rightarrow_{\beta} AAAA \rightarrow_{\beta} \dots$$

Nu se ajunge la o formă normală, pentru că mereu rămâne un redex de forma $(\lambda y.yyy)A$. Deci termenul nu este slab normalizabil. $\square$

6.8 Incluziunile $\rightarrow_{\beta} \subseteq \rightarrow_{\parallel}$ si $\rightarrow_{\parallel} \subseteq \rightarrow_{\beta}^*$

Enunț. Demonstrați că reducerea paralelă include reducerea beta într-un pas si este inclusă în reducerea beta în mai mulți pași:

$$\rightarrow_{\beta} \subseteq \rightarrow_{\parallel}, \quad \rightarrow_{\parallel} \subseteq \rightarrow_{\beta}^*.$$

Rezolvare. 1. $\rightarrow_{\beta} \subseteq \rightarrow_{\parallel}$. Inducție după regula care produce $M \rightarrow_{\beta} N$.

Dacă pasul este redexul principal,

$$(\lambda x.M)P \rightarrow_{\beta} M[x := P],$$

atunci regula de reducere paralelă pentru redex permite exact

$$(\lambda x.M)P \rightarrow_{\parallel} M[x := P],$$

luând $M \rightarrow_{\parallel} M$ si $P \rightarrow_{\parallel} P$ prin reflexivitatea paralelă.

Dacă beta-reducerea are loc sub λ , în stânga sau în dreapta unei aplicații, folosim ipoteza de inducție si regulile de compatibilitate ale reducerii paralele.

2. $\rightarrow_{\parallel} \subseteq \rightarrow_{\beta}^*$. Inducție după derivarea $M \rightarrow_{\parallel} N$.

Caz reflexiv: $M \rightarrow_{\parallel} M$, iar $M \rightarrow_{\beta}^* M$ prin reflexivitate.

Caz abstracție: dacă $M \rightarrow_{\parallel} M'$, atunci prin inducție $M \rightarrow_{\beta}^* M'$, deci $\lambda x.M \rightarrow_{\beta}^* \lambda x.M'$.

Caz aplicație: dacă $M \rightarrow_{\parallel} M'$ si $N \rightarrow_{\parallel} N'$, atunci prin inducție $M \rightarrow_{\beta}^* M'$ si $N \rightarrow_{\beta}^* N'$. Prin compatibilitatea lui $\rightarrow_{\beta}^*$ cu aplicația,

$$MN \rightarrow_{\beta}^* M'N'.$$

Caz redex paralel:

$$(\lambda x.M)N \rightarrow_{\parallel} M'[x := N']$$

cu $M \rightarrow_{\parallel} M'$ și $N \rightarrow_{\parallel} N'$. Prin inducție,

$$M \rightarrow_{\beta}^* M', \quad N \rightarrow_{\beta}^* N'.$$

Atunci

$$(\lambda x.M)N \rightarrow_{\beta}^* (\lambda x.M')N' \rightarrow_{\beta} M'[x := N'].$$

Deci $(\lambda x.M)N \rightarrow_{\beta}^* M'[x := N']$. □

6.9 Comutativitatea substituției

Enunț. Dacă $x \neq y$ și $x \notin \text{FV}(P)$, demonstrați:

$$M[x := N][y := P] = M[y := P][x := N[y := P]].$$

Ideea de examen. Ordinea substituțiilor poate fi schimbată dacă ajustăm termenul pus pentru x . Condiția $x \notin \text{FV}(P)$ previne reapariția lui x prin substituția lui y .

Rezolvare. Demonstrația complet formală se face prin inducție structurală după M . Redăm cazurile principale.

Caz $M = z$ **variabilă.**

Dacă $z = x$, stânga este

$$x[x := N][y := P] = N[y := P].$$

Dreapta este

$$x[y := P][x := N[y := P]] = x[x := N[y := P]] = N[y := P],$$

deoarece $x \neq y$.

Dacă $z = y$, stânga este

$$y[x := N][y := P] = y[y := P] = P.$$

Dreapta este

$$y[y := P][x := N[y := P]] = P[x := N[y := P]].$$

Cum $x \notin \text{FV}(P)$, substituția pentru x nu schimbă P , deci dreapta este P .

Dacă $z \neq x, y$, ambele părți rămân z .

Caz $M = M_1 M_2$. **Avem:**

$$(M_1 M_2)[x := N][y := P] = M_1[x := N][y := P] M_2[x := N][y := P].$$

Prin ipoteza de inducție pe M_1 și M_2 , aceasta este

$$M_1[y := P][x := N[y := P]] M_2[y := P][x := N[y := P]],$$

adică

$$(M_1 M_2)[y := P][x := N[y := P]].$$

Caz $M = \lambda z.M_0$. Dacă $z = x$ sau $z = y$, substituția corespunzătoare se oprește sub binderul respectiv. Dacă z este diferit de x, y și nu produce captură, aplicăm ipoteza de inducție în corp. Dacă ar produce captură, se face mai întâi α -redenomirea lui z cu o variabilă proaspătă. Cum lucrăm modulo α -echivalență, demonstrația continuă ca în cazul fără captură.

Prin inducție structurală, egalitatea este adevărată pentru orice termen M . □

6.10 Lemele de formă pentru reducerea paralelă

Enunț. Demonstrați:

- (a) dacă $\lambda x.M \rightarrow_{\parallel} N$, atunci $N = \lambda x.M'$ si $M \rightarrow_{\parallel} M'$;
 (b) dacă $MN \rightarrow_{\parallel} L$, atunci fie $L = M'N'$ cu $M \rightarrow_{\parallel} M'$ si $N \rightarrow_{\parallel} N'$, fie $M = \lambda x.P$ si $L = P'[x := N']$ cu $P \rightarrow_{\parallel} P'$ si $N \rightarrow_{\parallel} N'$.

Rezolvare. Acestea se demonstrează prin analiza ultimei reguli folosite în derivarea paralelă.

- (a) Pentru un termen de forma $\lambda x.M$, singura regulă ne-trivială de reducere paralelă care se poate aplica la rădăcină este regula pentru abstracție:

$$\frac{M \rightarrow_{\parallel} M'}{\lambda x.M \rightarrow_{\parallel} \lambda x.M'}.$$

Mai există cazul reflexiv, care este inclus luând $M' = M$. Deci rezultatul are forma $\lambda x.M'$ si $M \rightarrow_{\parallel} M'$.

- (b) Pentru un termen de forma MN , ultimele reguli posibile sunt:

- regula de aplicație:

$$\frac{M \rightarrow_{\parallel} M' \quad N \rightarrow_{\parallel} N'}{MN \rightarrow_{\parallel} M'N'};$$

atunci $L = M'N'$;

- regula de redex paralel, posibilă doar dacă partea stângă este o abstracție, adică $M = \lambda x.P$:

$$\frac{P \rightarrow_{\parallel} P' \quad N \rightarrow_{\parallel} N'}{(\lambda x.P)N \rightarrow_{\parallel} P'[x := N']}.$$

atunci $L = P'[x := N']$.

Acestea sunt exact cele două alternative din enunț. □

6.11 Substituția păstrează reducerea paralelă

Enunț. Demonstrați:

- (a) dacă $N \rightarrow_{\parallel} N'$, atunci $M[x := N] \rightarrow_{\parallel} M[x := N']$;
 (b) dacă $M \rightarrow_{\parallel} M'$ si $N \rightarrow_{\parallel} N'$, atunci $M[x := N] \rightarrow_{\parallel} M'[x := N']$.

Rezolvare. (a) Inducție structurală după M .

Dacă $M = x$, atunci $M[x := N] = N$ si $M[x := N'] = N'$, deci concluzia este chiar ipoteza $N \rightarrow_{\parallel} N'$.

Dacă $M = y \neq x$, ambele substituții dau y , iar $y \rightarrow_{\parallel} y$ prin reflexivitate.

Dacă $M = M_1M_2$, aplicăm ipoteza de inducție pe M_1 si M_2 , apoi regula paralelă pentru aplicație.

Dacă $M = \lambda y.P$, alegem reprezentant α -echivalent fără captură. Dacă $y = x$, substituția nu intră sub binder si avem reflexivitate. Dacă $y \neq x$, aplicăm inducția pe corpul P , apoi regula pentru abstracție.

- (b) Inducție după derivarea $M \rightarrow_{\parallel} M'$.

Caz reflexiv: dacă $M' = M$, concluzia devine partea (a).

Caz abstracție si aplicație: se aplică ipoteza de inducție pe premise si regulile paralele corespunzătoare.

Caz redex paralel: dacă

$$M = (\lambda y.P)Q, \quad M' = P'[y := Q'],$$

cu $P \rightarrow_{\parallel} P'$ si $Q \rightarrow_{\parallel} Q'$, atunci după substituția $[x := N]$ se reduce paralel la același rezultat ca $M'[x := N']$, folosind comutativitatea substituției si ipotezele de inducție pe P, Q . Lucrăm modulo α -echivalență pentru a evita capturarea variabilelor.

Deci substituția este compatibilă cu reducerea paralelă. □

6.12 De ce Church-Rosser dă unicitatea formei normale

Enunț. Folosind teorema Church-Rosser, demonstrați că un lambda-termen are cel mult o formă normală, modulo α -echivalență.

Rezolvare. Presupunem că M are două forme normale N_1 si N_2 :

$$M \rightarrow_{\beta}^* N_1, \quad M \rightarrow_{\beta}^* N_2,$$

unde N_1 si N_2 sunt forme normale.

Prin Church-Rosser, există un termen P astfel încât

$$N_1 \rightarrow_{\beta}^* P, \quad N_2 \rightarrow_{\beta}^* P.$$

Dar N_1 este formă normală, deci nu mai are niciun pas β posibil. Singura reducere în mai mulți pași din N_1 este cea de lungime zero, deci $P \equiv_{\alpha} N_1$. Analog, $P \equiv_{\alpha} N_2$.

Prin tranzitivitatea α -echivalenței,

$$N_1 \equiv_{\alpha} N_2.$$

Deci forma normală este unică modulo α -echivalență. □

6.13 Proprietăți ale reducerii standard

Enunț. Demonstrați proprietățile:

- (a) $M \rightarrow_{st} M$;
- (b) dacă $M \rightarrow_h^* N$, atunci $MP \rightarrow_h^* NP$;
- (c) dacă $L \rightarrow_h^* M \rightarrow_{st} N$, atunci $L \rightarrow_{st} N$;
- (d) dacă $M \rightarrow_h^* N$, atunci $M[x := P] \rightarrow_h^* N[x := P]$;
- (e) dacă $M \rightarrow_{st} N$ si $P \rightarrow_{st} Q$, atunci $M[x := P] \rightarrow_{st} N[x := Q]$.

Rezolvare. (a) Din definiția reducerii standard, orice termen se reduce standard la el însuși prin secvență vidă sau prin cazul reflexiv generat de reducerea head de lungime zero.

(b) Reducerea head are loc în poziția cea mai din stânga-exterioară. Dacă M face reduceri head până la N , atunci în aplicația MP , aceleași reduceri se pot face în partea stângă, iar argumentul P rămâne neschimbat. Deci $MP \rightarrow_h^* NP$.

(c) Dacă $L \rightarrow_h^* M$ si apoi $M \rightarrow_{st} N$, definiția lui $\rightarrow_{st}$ permite prefixarea unei secvențe standard cu reduceri head. Deci $L \rightarrow_{st} N$.

(d) Demonstrație prin inducție după lungimea secvenței $M \rightarrow_h^* N$. Pentru lungime zero e trivial. Pentru un pas head $M \rightarrow_h M_1$, substituția păstrează poziția de head-redex, eventual după α -redenumire, deci $M[x := P] \rightarrow_h M_1[x := P]$. Continuăm cu inducția.

(e) Inducție după derivarea standard $M \rightarrow_{st} N$. În cazurile structurale, folosim ipoteza de inducție pe subtermeni si faptul că $P \rightarrow_{st} Q$. Cazul aplicației se combină cu definiția standard pe aplicații; cazul abstracției cu definiția standard sub λ . Cazul head folosește (d). Rezultatul este că putem substitui termeni care reduc standard si obținem tot o reducere standard. $\square$

6.14 Lema: dacă $L \rightarrow_{st} (\lambda x.M)N$, atunci $L \rightarrow_{st} M[x := N]$

Rezolvare. Dacă $L \rightarrow_{st} (\lambda x.M)N$, atunci prin definiția reducerii standard, există o secvență standard care ajunge la o aplicație al cărei stâng este o abstracție. Aplicarea redexului principal

$$(\lambda x.M)N \rightarrow_{\beta} M[x := N]$$

este exact o reducere head. Proprietățile reducerii standard spun că o secvență standard poate fi continuată cu o reducere head si rămâne standard. Deci

$$L \rightarrow_{st} M[x := N].$$

Mai explicit: secvența standard de la L la $(\lambda x.M)N$ a redus doar redexuri care apar înainte sau în poziții compatibile. Redexul principal obținut este acum cel mai din stânga-exterior, deci poate fi redus fără a strica standardizarea. $\square$

6.15 Absorbția unei beta-reduceri într-o reducere standard

Enunț. Dacă $L \rightarrow_{st} M$ si $M \rightarrow_{\beta} N$, atunci $L \rightarrow_{st} N$.

Rezolvare. Demonstrația se face prin inducție după regula care produce pasul $M \rightarrow_{\beta} N$.

Cazul redex principal. Dacă

$$M = (\lambda x.P)Q, \quad N = P[x := Q],$$

si $L \rightarrow_{st} (\lambda x.P)Q$, atunci lema precedentă dă direct

$$L \rightarrow_{st} P[x := Q] = N.$$

Cazul sub abstracție. Dacă $M = \lambda x.P$, $N = \lambda x.P'$ si $P \rightarrow_{\beta} P'$, din $L \rightarrow_{st} \lambda x.P$ rezultă, prin definiția reducerii standard, că se ajunge standard sub abstracție. Aplicăm ipoteza de inducție pe corp si reconstruim abstracția.

Cazul în stânga unei aplicații. Dacă $M = PQ$, $N = P'Q$, cu $P \rightarrow_{\beta} P'$, descompunem reducerea standard a lui L până la PQ conform regulilor pentru aplicație. Aplicăm ipoteza de inducție pe partea stângă si păstrăm argumentul.

Cazul în dreapta unei aplicații. Similar, dar se folosește faptul că în secvența standard redexurile din stânga au fost tratate înaintea celor din dreapta; reducerea în dreapta poate fi absorbită în componenta standard a argumentului.

Prin urmare orice pas beta făcut după o reducere standard poate fi inclus într-o reducere standard mai lungă. $\square$

6.16 Teorema de standardizare - demonstrație de examen

Enunț. Dacă $M \rightarrow_{\beta}^* N$, atunci există o secvență standard de β -reducție de la M la N . În particular, dacă M are formă normală, strategia normală o poate găsi.

Rezolvare. Demonstrația se face prin inducție după lungimea secvenței $M \rightarrow_{\beta}^* N$.

Dacă lungimea este 0, atunci $M = N$, iar secvența vidă este standard.

Pentru pasul inductiv, presupunem

$$M \rightarrow_{\beta}^* K \rightarrow_{\beta} N.$$

Prin ipoteza de inducție, există o secvență standard

$$M \rightarrow_{st} K.$$

Prin lema de absorbție, din $M \rightarrow_{st} K$ și $K \rightarrow_{\beta} N$ rezultă

$$M \rightarrow_{st} N.$$

Deci orice secvență de reduceri beta poate fi rearanjată într-una standard.

Pentru consecința despre strategia normală: dacă M are formă normală N , există o secvență $M \rightarrow_{\beta}^* N$. Prin standardizare, există o secvență standard de la M la N . Strategia normală reduce mereu redexul cel mai din stânga dintre cele exterioare; o astfel de strategie este standard. Deci, dacă o formă normală există, reducerea normală nu o ratează. $\square$

Checklist final pentru examen

Subiect	Ce trebuie să poți scrie rapid
Funcții și grafice	Domeniu, codomeniu, grafic, proprietatea funcțională, inversabilă iff bijectivă.
Recursie structurală	Bază, cazuri inductive, unicitate prin compararea a două funcții.
Big-step IMP	Reguli pentru expresii, instrucțiuni, determinism, while true/false.
Hoare	Atribuire lucrată invers, secvențiere, if, while cu invariant, întărire pre, slăbire post.
wp/ws/wls	Definiții, echivalența cu tripletul Hoare, diferența între terminare și neterminare.
Unificare	SCOATE, DESCOMPUNE, REZOLVĂ, conflict, ciclu, mgu, invariant, variantă.
SLD	Scop, clauză, cgu, arbore SLD, respingere, substituție calculată.
Lambda	FV, substituție fără captură, α , β , formă normală, Church-Rosser, standardizare.